



4.2.1 Data structures

Revision pack Dec 2021

Name: _____

Class: _____

Date: _____

Time: **347 minutes**

Marks: **237 marks**

Comments:

Q1.

Explain how the elements in a non-empty queue may be reversed with the aid of a stack.

(Total 4 marks)

Q2.

A computer simulation is to be used to imitate the flow of students through a school canteen. The simulation will be based on a model developed by the school's canteen manager and a Computing student.

- (a) In the context of simulation, explain what a model is.

(1)

- (b) Students must queue at a particular serving point in the canteen if they wish to purchase hot food.

The Computing student intends to represent the queue of students waiting to be served as a dynamic data structure using a linked list.

- (i) Explain what pointers the student will need to create and what they will be used for.

(2)

- (ii) Teachers are able to bypass the students in the queue by walking past them. However, a teacher may not always go directly to the very front of the queue as it may contain teachers already. In which case, the teacher joins the queue at the point just behind the other teachers.

What type of queue would the Computing student use to represent this situation?

(1)

- (c) The Computing student decides that she will need to use the random number generator in the programming language that she is using to develop the simulation.

Give **one** example of something that she might need to use random numbers for when producing this simulation.

(1)
(Total 5 marks)

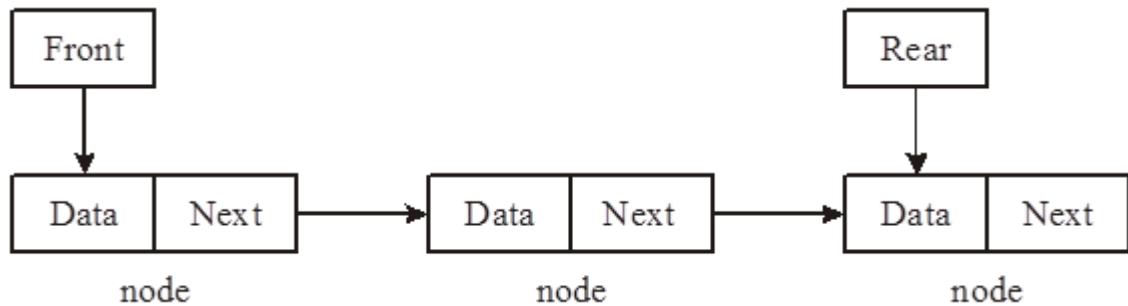
Q3.

- (a) Draw a diagram to show the structure of a queue.

(3)

- (b) Write an algorithm to show how a data item is added to a queue. Take into account the possibility that the queue is full.

Q4.



- (a) Assume a queue is implemented as a linked list using pointers as in the figure.

Give the **three** steps required to remove a node from the front of the queue and recover the memory space occupied by the node.

1. _____
2. _____
3. _____

(3)

- (b) A set of operations are defined to manipulate the contents of the queue. As well as *Remove* these include *FrontItem* and *IsQueueEmpty*.

Name another operation that would be essential to use this queue.

(1)

- (c) The queue could be implemented using an array instead of a linked list.

- (i) What additional operation will be required if the queue is implemented using an array?

(1)

- (ii) Give **one** advantage of array implementation.

(1)

- (iii) Give **two** disadvantages of array implementation.

1. _____

2. _____

Q5.

(a) In the context of data structures what is meant by the terms:

(i) FIFO; _____

(ii) LIFO? _____

(2)

(b) Queue and stack are examples of data structures. Tick in the following table to indicate whether they are FIFO or LIFO data structures.

	FIFO	LIFO
Queue		
Stack		

(2)

(c) Describe **one** example of the use of a stack.

(2)

(d) Describe **one** example of the use of a Binary Search Tree.

(2)

(Total 8 marks)

Q6.

The following data is input to a program, in alphabetical order, and is stored.

Anne
Bob
Claire
Dean

(a) Draw a diagram to show how this data is stored for:

(i) a stack;

(ii) a queue.

(4)

(b) One item is retrieved from these data structures for processing, and Eden is input.

Draw the diagrams of this new situation for:

(i) the stack;

(ii) the queue.

(3)

(c) Why are queues in computer systems usually implemented as circular queues?

(2)

(Total 9 marks)

Q7.

A stack may be implemented by using either an array or a linked list.

(a) Give a disadvantage of:

(i) an array implementation;

_____ (1)

(ii) a linked list implementation.

_____ (1)

(b) Under what circumstances would it be more appropriate to use:

(i) an array;

_____ (1)

(ii) a linked list.

_____ (1)

(Total 4 marks)

Q8.

The data shown below is a list of surnames of 20 motor car policyholders with the number of claims they have each made in the last five years.

PolicyHolder		NoOfClaims	
1	Wilcox	1	1
2	Adams	2	0
3	Pollard	3	0
4	Williams	4	0
5	Searle	5	3
6	Kelly	6	0
7	Lewis	7	1
8	Franks	8	5
9	Patel	9	1
10	Li Che	10	0
...		...	...
...		...	...

19	Wilkinson	19	3
20	Veale	20	0

- (a) (i) The user inputs a policyholder. If the surname is found, the program outputs the number of claims for that policyholder.

```

Read(SearchName)
For P := 1 To 20 Do
    If PolicyHolder[P] = SearchName
        Then GoTo 200
    GoTo 300
200 : Write(NoOfClaims[P])
300: End

```

Give **two** reasons why this is badly designed program code.

1. _____

2. _____

(2)

- (ii) Write declaration statements (in a language with which you are familiar) for the PolicyHolder or NoOfClaims data structure above, and one other variable used in the code above.

The programming language I am using is _____

1. _____

2. _____

(2)

- (b) A new task is to design and write code to establish if there are any policyholders who have made five or more claims. The program will output a 'yes' or 'no' message only.

Write the code for this new task in a programming language with which you are familiar.

(Hint: Use a loop structure to initiate the loop, and then end the loop when some condition is met.)

(5)
(Total 9 marks)

Q9.

The list Ports contains the following names:

[Southampton, Barcelona, Athens, Alexandria, Tunis, Lisbon]

The table below shows some functions which take a list as their single argument and return a result which is either an element of a list or a boolean value.

Head(list) – If the list is non-empty, it returns the element at the head of the list (e.g. Head (Ports) → Southampton) otherwise it reports an error
Tail(list) – If the list is non-empty it returns a new list containing all but the first element of the original list, otherwise it reports an error
Empty(list) – if the list is the empty list it returns True otherwise it returns False. The empty list is donated by []

(a) What result is returned when the following function calls are made?

(i) Tail(Ports) _____

(1)

(ii) Head(Tail(Tail(Ports))) _____

(2)

(iii) Empty(Tail(Tail(Tail(Tail(Tail(Tail(Ports))))))) _____

(2)

A recursively defined procedure P, which takes a list as its single parameter, is defined below.

```

Define Procedure P(list)
  If Not Empty(list)
    Then
      P(Tail(list))
      Print Head(list)
    EndIf
  EndDefine

```

(b) What is meant by recursively defined?

(1)

(c) Explain why a stack is needed to execute procedure P recursively.

(2)

(d) For the procedure call P(Ports) give the PRINTed output in the order in which it is produced.

(4)

(e) Complete the table to show the list Ports as a linked list so that the ports can be accessed in alphabetical order.

1	Southampton	
2	Barcelona	
3	Athens	
4	Alexandria	
5	Tunis	
6	Lisbon	

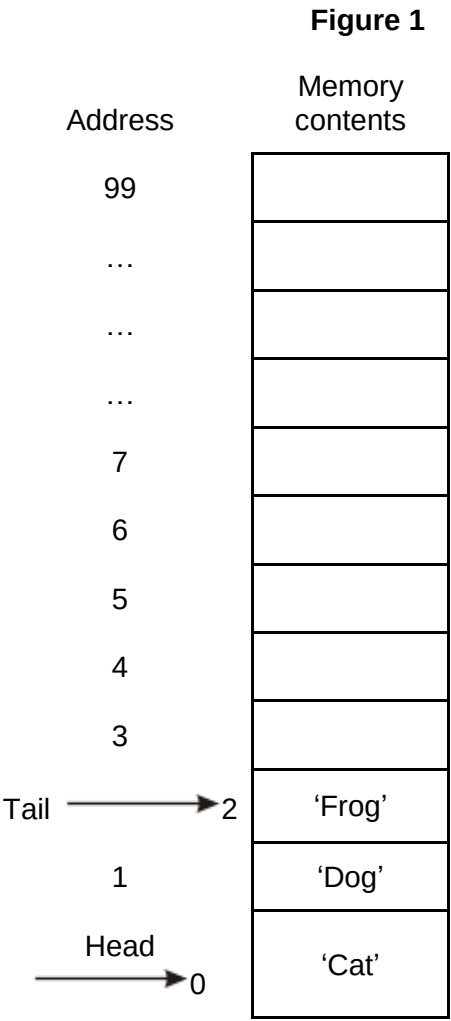
Head Pointer

(2)

(Total 14 marks)

Q10.

Figure 1 shows an area of 100 memory locations which are used to store string data values. This area of memory behaves as a queue. The Head and Tail values are used to control changes to the queue. **Figure 1** shows the state of the queue after three values have been added.

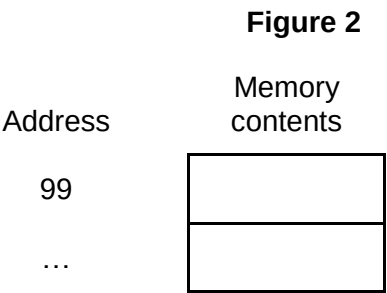


(a) What is the function of the Tail pointer?

(1)

(b) Which item in the queue will be the first item to leave?

(1)



...	
...	
7	
6	
5	
4	
3	
2	'Frog'
1	'Dog'
0	'Cat'

- (c) Three new items join the queue in the order 'Snake', 'Eel' and 'Shark' and two items then leave.

Draw on **Figure 2** the new state of the queue, including the Head and Tail pointers.

(3)

- (d) After extensive data changes the queue will become unusable. Explain why this is so.

(2)

(Total 7 marks)

Q11.

A retail store employs ten sales staff. Staff try to persuade customers to take out a store card with the company when they make a purchase. The store keeps a record of the number of new store cards issued by its sales staff over the first six months of the year.

Table 1

StoreCards						
	[1]	[2]	[3]	[4]	[5]	[6]
[1]	12	12	6	8	3	2
[2]	12	17	7	4	5	6
[3]	2	12	0	12		
[4]	4	10	7	4		

[5]	5	0	0	0	0	0
[6]	6	1	4	6	7	8
[7]	12	19	12	16	17	6
[8]	13	9	7	3	4	5
[9]	12	8	4	4	5	4
[10]	14	11	12	4	5	6

The data is to be stored in a 2-dimensional array with identifier StoreCards as shown in the table above. The first subscript of the array represents the row number (the salesperson number), and the second subscript the column number (the month).

- (a) In the table the value 16 has been **emboldened**. Explain what this value represents.

(2)

- (b) Write a declaration statement for the array StoreCards.

(2)

- (c) Using the data given in the table above, write an assignment statement for the January sales for salesperson 8.

(2)

- (d) Study the pseudo-code below.

```

Input SalesPersonNumber
PersonTotal ← 0
For Month ← 1 to 6 Do
    PersonTotal ← PersonTotal +
        storeCards[SalesPersonNumber, Month]
End For
Print PersonTotal

```

Explain what this algorithm is designed to do.

(2)

- (e) A number of programs are to be written for the store card application, and the

following are some of the data values which will need to be stored and/or calculated.

State what data type the programmer would use for each data item below.

- (i) Average overtime hours worked by each member of staff.

(1)

- (ii) Whether or not the staff are willing to work on Boxing Day.

(1)

- (iii) The number of customer complaints made about each member of staff.

(1)

(Total 11 marks)

Q12.

Explain the differences between static and dynamic data structures.

(Total 2 marks)

Q13.

A *linear search* and a *binary search* are **two** different methods of searching an ordered list. A given list contains 271 items.

- (a) (i) What is the maximum number of items accessed when searching for a particular item from the given list using a linear search?

(1)

- (ii) Explain your answer.

(1)

- (b) (i) What is the maximum number of items accessed when searching for a particular item from the given list using a binary search?

(1)

(ii) Explain your answer.

(1)

(c) An integer array A contains the following items.

A	
[1]	23
[2]	45
[3]	16
[4]	12
[5]	31

(i) Dry run the following algorithm by completing the trace table.

```

For Count1 ← 1 To 4
    For Count2 ← 1 To 4
        If A[Count2] > A[Count2 + 1] Then
            Temp ← A [ Count2]
            A[Count2] ← A[Count2 + 1]
            A[Count2 + 1] ← Temp
        EndIf
    EndFor
EndFor

```

Count 1	Count2	Temp	A				
			[1]	[2]	[3]	[4]	[5]
			23	45	16	12	31
1	1						

(5)

(ii) What is the purpose of this algorithm?

(1)

(iii) Suggest **one** way the algorithm could be improved.

(1)

(Total 11 marks)

Q14.

A *recursively-defined* procedure **ProcA** that takes two integers as parameters is defined below.

(a) What is meant by a recursively-defined procedure?

(1)

(b) What is the role of the stack when a recursively-defined procedure is executed?

(1)

(c) Dry run the procedure call **ProcA(11,1)** using the data in the array, **Items**, by completing the trace table below.

		Items	
Procedure ProcA	(Number,Entry)	[1] <table><tr><td>4</td></tr></table>	4
4			


```

    If Number <> Items[Entry]
    Then ProcA (Number,Entry+1)
    Else Output (Entry)
    EndIf
EndProc

```

[2]	5
[3]	8
[4]	11
[5]	15
[6]	19
[7]	21
[8]	28
[9]	33

Number	Entry	Output
11	1	

(4)

- (d) What is the purpose of this algorithm?

(1)

- (e) Give a situation where this algorithm will fail.

(1)

- (f) Suggest a modification to the algorithm that will prevent it from failing.

(1)

- (g) With an ordered array, Items, of many more entries, what more efficient algorithm could be used to achieve your expressed purpose in part (d)?

(1)

(Total 10 marks)

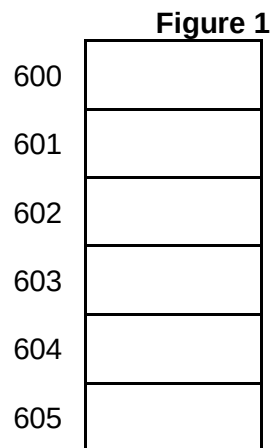
Q15.

- (a) State the principle of operation of a set of data values which behave as a stack.

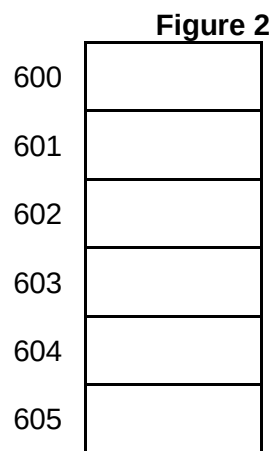
(1)

- (b) Memory locations 600 to 605 are to be used as a stack area to store character data, and the first value added to the stack is to be stored at address 600.

Figure 1 shows the initial empty state of the stack.

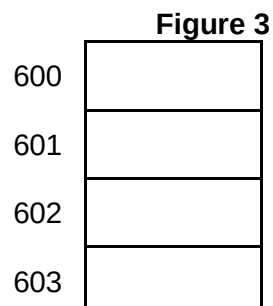


- (i) Show on **Figure 2** the state of the stack after the characters 'A', 'V', 'E', 'R' and 'Y' join the stack.



(1)

- (ii) Two items are removed from the stack. Show on **Figure 3** the state of the stack.



604	
605	

(1)

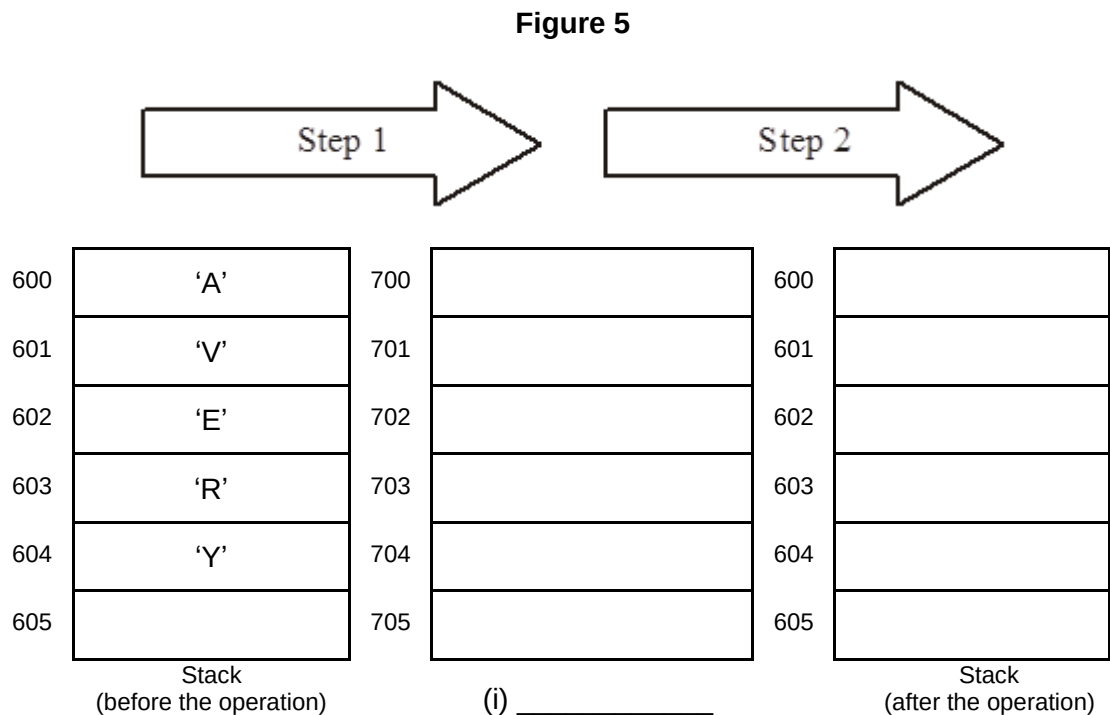
- (iii) Two new characters 'S' and 'P' join the stack. Show on **Figure 4** the final state of the stack.

Figure 4

600	
601	
602	
603	
604	
605	

(1)

- (c) The original items in this stack are to be reversed. This can be done using a second data structure which uses locations 700 to 705 respectively. The first item added to the stack was character 'A'.



- (i) Name the second data structure. Label **Figure 5**.

(1)

- (ii) Describe **Step 1** in **Figure 5**.

(1)

(iii) Describe **Step 2** in **Figure 5**.

(1)

(iv) Show on **Figure 5** the final contents of all the memory locations.

(2)
(Total 9 marks)

Q16.

A firm selling double glazing employs three sales staff. Each person is given a sales target for each of the four quarters of the year.

- Quarter 1 January – March
- Quarter 2 April – June
- Quarter 3 July – September
- Quarter 4 October – December

Based on all the sales made, the data in **Table 1** is produced showing whether or not each sales person achieved their target sales for each quarter. Each value is stored as a single character 'Y' (sales target met) or 'N' (sales target not met).

The columns represent each quarter, each row represents a salesperson.

Table 1

	Target			
	[1]	[2]	[3]	[4]
[1]	Y	N	Y	N
[2]	N	N	Y	Y
[3]	N	N	N	N

(a) What data structure could be used in a programming language for organising the data shown in **Table 1**?

(1)

(b) One of the data values in **Table 1** has been emboldened. What does this value represent?

(1)

(c) The following algorithm processes the data shown in **Table 1**. Trace the execution

of the algorithm by completing **Table 2**.

```

For Quarter ← 1 To 4
  Do NewArray [Quarter] ← 0
End For
For Person ← 1 To 3
  Do
    For Quarter ← 1 To 4
      Do
        If Target[Person, Quarter] = 'N'
          Then NewArray [Quarter] ← NewArray [Quarter] + 1
        End If
      End For
    End For
  End For
End For

```

Table 2

Person	Quarter	Target[Person, Quarter]	NewArray			
			[1]	&nb	[3]	[4]
	1					

(6)

(d) Explain what numbers are being calculated and stored in the `NewArray` data structure.

(2)
(Total 10 marks)

Q17.

A queue may be implemented by using either an array or a linked list.

(a) Give a disadvantage of

(i) an array implementation.

(1)

(ii) a linked list implementation.

(1)

(b) As items are added and removed in the array implementation the queue will gradually move along the array. How can the program deal with the situation when the end of the array is reached?

(1)

(c) A queue is implemented with the following operations:

AddItem – add an item to the queue

RemoveItem – remove an item from the queue

FrontItem – obtain the item at the front of the queue

IsEmpty – return true if the queue is empty, otherwise return false.

What additional operation is required if the queue is implemented using an array?

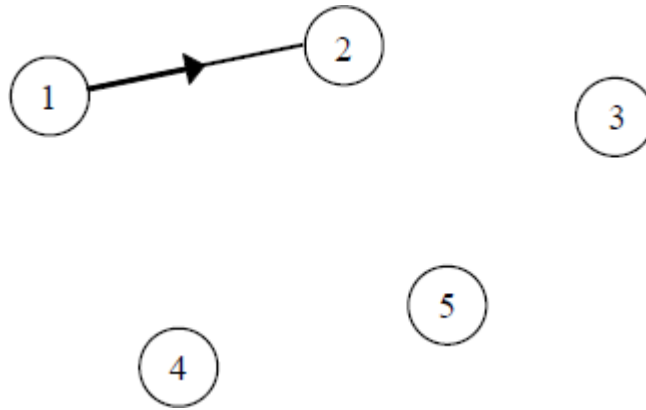
(1)
(Total 4 marks)

Q18.

The table below shows an adjacency matrix representation of a directed graph (digraph).

		1	2	3	4	5
From	1	0	1	0	1	0
	2	0	0	1	1	0
	3	0	0	0	0	0
	4	0	0	0	0	1
	5	0	1	0	0	0

- (a) Complete this unfinished diagram of the directed graph.



(2)

- (b) Directed graphs can also be represented by an adjacency list.

Explain under what circumstances an adjacency matrix is the most appropriate method to use to represent a directed graph, and under what circumstances an adjacency list is more appropriate.

(2)

- (c) A tree is a particular type of graph.

What properties must a graph have for it to be a tree?

(2)

- (d) Data may be stored as a binary tree.

Show how the following data may be stored as a binary tree for subsequent processing in alphabetic order by drawing the tree. Assume that the first item is the root of the tree and the rest of the data items are inserted into the tree in the order given.

Data items: Jack, Bramble, Snowy, Butter, Squeak, Bear, Pip

(3)

- (e) A binary tree such as the one created in part (d) could be represented using one array of records or, alternatively, using three one-dimensional arrays.

Describe how the data stored in the array(s) could be structured for **one** of these two possible methods of representation.

(3)

(Total 12 marks)

Q19.

A list data structure can be represented using an array.

The pseudocode algorithm in the diagram below can be used to carry out one useful operation on a list.

```
p ← 1
If ListLength > 0 Then
    While p <= ListLength And List [p]
        p ← p + 1
    EndWhile
    For q ← ListLength DownTo p Do
        List[q + 1] := List[q]
    EndFor
EndIf
List[p] ← New
ListLength ← ListLength + 1
```

- (a) The initial values of the variables for one particular execution of the algorithm are shown in the trace table below.

Complete the trace table for the execution of the algorithm.

				List				
ListLength	New	p	q	[1]	[2]	[3]	[4]	[5]
4	38	-	-	9	21	49	107	

(4)

(b) Describe the purpose of the algorithm the diagram above.

(1)

(c) A list implemented using an array is a static data structure. The list could be implemented using a linked list as a dynamic data structure instead.

(i) Describe **one** difference between a static data structure and a dynamic data structure.

(1)

(ii) If the list were to be implemented as a dynamic data structure, explain what the heap would be used for.

(1)

(Total 7 marks)

Q20.

Reverse Polish Notation is an alternative to standard infix notation for writing arithmetic expressions.

- (a) Convert the following Reverse Polish Notation expressions to their equivalent infix expressions.

Reverse Polish Notation	Equivalent Infix Expression
45 6 +	
12 19 + 8 *	

(2)

- (b) State **one** advantage of Reverse Polish Notation over infix notation.

(1)

- (c) The pseudo-code algorithm below can be used to calculate the result of evaluating a Reverse Polish Notation expression that is stored in a string. The algorithm is designed to work only with the single digit denary numbers 0 to 9. It uses procedures and functions listed in the table below, two of which operate on a stack data structure.

```

StringPos ← 0
Repeat
  StringPos ← StringPos + 1
  Token ← GetCharFromString(InputString, StringPos)
  If Token = '+' Or Token = '-' Or Token = '/' Or Token = '*'
  Then
    Op2 ← Pop()
    Op1 ← Pop()
    Case Token Of
      '+': Result ← Op1 + Op2
      '-': Result ← Op1 - Op2
      '/': Result ← Op1 / Op2
      '*': Result ← Op1 * Op2
    EndCase
    Push(Result)
  Else
    IntegerVal ← ConvertToInteger(Token)
    Push(IntegerVal)
  EndIf
Until StringPos = Length(InputString)
Output Result

```

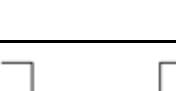
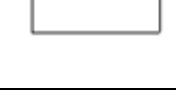
Procedure/Function	Purpose	Example(s)
GetCharFromString (InputString:String, StringPos:Integer): Char	Returns the character at position StringPos within the string InputString. Note that the leftmost letter is position 1, not position 0.	GetCharFromString("Computing", 1) would return the character 'C'. GetCharFromString("Computing", 3) would return the character 'm'.
ConvertToInteger (ACharacter: Char):	Returns the integer equivalent of the	ConvertToInteger('4') would return the

Integer	character in ACharacter.	integer value 4.
Length (AString: String): Integer	Returns a count of the number of characters in the string AString.	Length("AQA") would return the integer value 3.
Push (ANumber: Integer)	Puts the number in ANumber onto the stack.	Push(6) would put the number 6 on top of the stack.
Pop (): Integer	Removes the number from the top of the stack and returns it.	X ← Pop() would remove the value from the top of the stack and put it in x.

- (d) Complete the table below to trace the execution of the algorithm when
InputString is the string: 64+32+*

In the *Stack* column, show the contents of the stack once for each iteration of the
Repeat..Until loop, as it would be at the end of the iteration.

The first row and the leftmost column of the table have been completed for you.

StringPos	Token	IntegerVal	Op1	Op2	Result	Stack
0	-	-	-	-	-	
1						
2						
3						
4						

5						
6						
7						

(5)

Final output of algorithm: _____

(1)

- (e) A programmer is going to implement the algorithm above in a programming language that does not provide built-in support for a stack data structure.

The programmer intends to simulate a stack by using a fixed length array of 20 integers named `StackArray` with indices running from 1 to 20 and an integer variable `TopOfStackPointer` which will be initialised to 0.

Write a pseudo-code algorithm for the `Push` operation to push a value stored in the variable `ANumber` onto the stack.

Your algorithm should cope appropriately with any potential errors that might occur.

(4)

(Total 13 marks)

Q21.

A list data structure can be represented using a fixed-size array.

The pseudo-code algorithm in **Figure 1** can be used to carry out one useful operation on a list.

Figure 1

```
p ← 1
IF ListLength > 0 THEN
    WHILE p <= ListLength AND List[p] < New
        p ← p + 1
    ENDWHILE
    FOR q ← ListLength DOWNTO p DO
        List[q + 1] ← List[q]
    ENDFOR
ENDIF
List[p] ← New
ListLength ← ListLength + 1
```

The `DOWNTO` command causes the loop counter value to decrease by one on each iteration of the `FOR` loop.

The initial values of the variables for one particular execution of the algorithm are shown in the trace table opposite, labelled **Table 1**. Array indexing starts at 1.

(a) Complete the trace table for the execution of the algorithm in **Figure 1**.

Table 1

ListLength	New	p	q	List				
				[1]	[2]	[3]	[4]	[5]
4	48	–	–	19	43	68	107	–

(4)

(b) Describe the purpose of the algorithm in **Figure 1**.

(1)

- (c) A list may be implemented using a static data structure such as a fixed-length array, or using a dynamic data structure such as a linked list.

If the list were to be implemented using a dynamic data structure explain what the heap would be used for.

(1)

(Total 6 marks)

Q22.

A computer games programmer is writing a game. One aspect of the game involves a character who can carry various items, such as a bag of seeds, and an axe, around with her. The list of items that the character is currently carrying will be stored as a linked list of items of the `String` data type. The list is stored in no particular order.

The game is being developed using object-oriented programming. The **LinkedList** class will be used to store that list of items.

The class definition for the **LinkedList** class is:

```
LinkedList
= Class
  Public
    Procedure CreateList
    Procedure DestroyList
    Procedure AddItem(NewItem: String)
    Procedure DeleteItem(DelItem: String)
    Function ContainsItem(SearchItem: String): Boolean
    Function IsEmpty: Boolean
  Private
    Start: Pointer
    Current: Pointer
    Previous: Pointer
  End
```

- (a) Creating a class such as the **LinkedList** class, that can be used by other parts of a much bigger program, is a form of abstraction.

Explain why the **LinkedList** class is a form of abstraction.

(1)

- (b) Explain why the functions and procedures, such as `AddItem` have been declared to be `Public` whilst the data items such as `Start` have been declared as `Private`.

(2)

You may assume that:

- [illegible]

Q23.

A dictionary is an abstract data type that allows pairs of values to be associated with each other. For example, an English-French dictionary might associate English words with their translations into French. In such a dictionary, "Apple" would be associated with "Pomme" because "Pomme" is the French word for "Apple".

At a lower level of abstraction, a dictionary could be implemented as a data structure using a number of different methods. Two possible implementation methods are:

- **Implementation One:** As an unordered list in an array.
- **Implementation Two:** Using hashing, with the English word being passed through a hash function to calculate the position of the correct French translation in an array.

In each implementation, a record containing the English word and the equivalent French word are stored at each index in the array that is in use.

The figure below shows how an English-French dictionary containing five words could be implemented using these two methods.

Implementation One – Unordered List			Implementation Two – Hashing		
Index	English Word	French Word	Index	English Word	French Word
[1]	Apple	Pomme	[1]	Pear	Poire
[2]	Lemon	Citron	[2]	Lemon	Citron
[3]	Strawberry	Fraise	[3]		
[4]	Grapefruit	Pamplemousse	[4]		
[5]	Pear	Poire	[5]		
[6]			[6]		
[7]			[7]	Apple	Pomme
[8]			[8]	Strawberry	Fraise
[9]			[9]		
[10]			[10]	Grapefruit	Pamplemousse
New words are inserted into the array in the first available slot. The next word would be stored at position 6.			The position to store a word is calculated from the English word using a hashing function.		

- (a) Explain why, when the French translation of an English word needs to be looked up, **Implementation Two** is more time efficient than **Implementation One**.
-

(2)

- (b) In **Implementation Two**, it is possible that the hash function could compute the same value for two different English words.

Explain what the effect of this would be, and how it could be dealt with.

(2)

- (c) In **Implementation Two**, both the English and French words are stored at each index in the Array. In this implementation, explain why it would not be possible to perform reliable English to French translation if only the French words were stored.

(1)

(Total 5 marks)

Q24.

An interactive operating system maintains a list of the processes that are currently waiting to execute (run). The processes are stored in order of the priority that is associated with their execution. This priority can be set as "High", "Normal" or "Low".

Figure 1 and **Figure 2** below show two different ways in which the storage of the process list could be implemented.

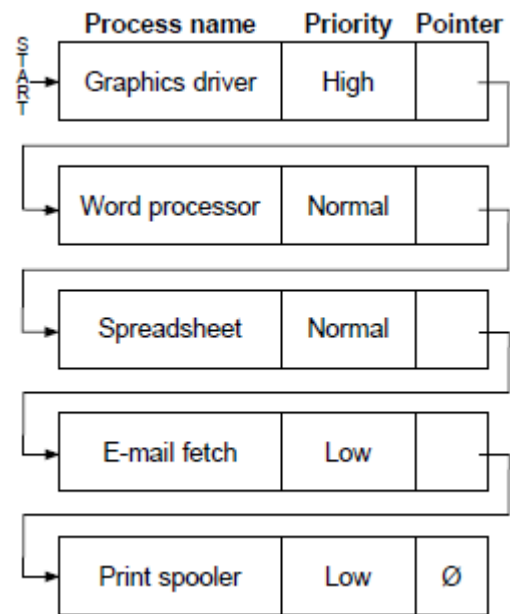
Figure 1

Figure 2

Static implementation: as an ordered list using a fixed size array.

Index	Process name	Priority
[1]	Graphics driver	High
[2]	Word processor	Normal
[3]	Spreadsheet	Normal
[4]	E-mail fetch	Low
[5]	Print spooler	Low
⋮		
[100]		

Dynamic implementation: as a linked list using dynamic memory allocation.



The process at the start of the list will be run next. In **Figure 1** and **Figure 2**, this is the "Graphics driver" process.

When a new process is initiated it is inserted into the list immediately after the last process of the same priority. A "Computer game" process with "High" priority would be inserted into the list in **Figure 1** and **Figure 2** between the "Graphics driver" and "Word processor" processes.

When a process is completed it is deleted from the list.

- (a) Explain **two** differences between a dynamic data structure and a static data structure.

Difference 1: _____

Difference 2: _____

(2)

- (b) The **static implementation** is less efficient at inserting new items into the list than the **dynamic implementation**.

Explain why this is the case.

(2)

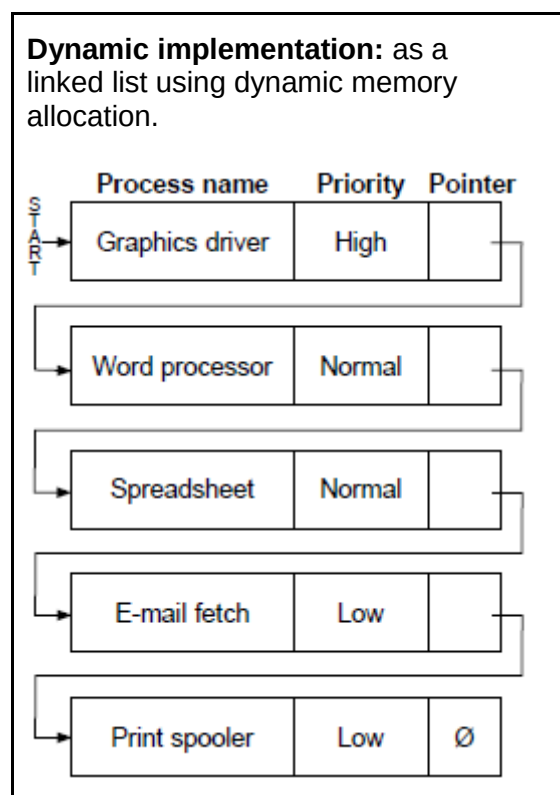
- (c) At a higher level of abstraction, the process list maintained by the operating system could be viewed as a type of queue.

What type of queue?

(1)

Figure 2 is repeated below so that you can answer the remaining question parts without having to turn back in the question booklet.

Figure 2 (repeated)



- (d) Consider the **dynamic implementation** in **Figure 2**.

- (i) What will the heap be used for in this implementation?
-
-

(1)

- (ii) In **Figure 2** pointers are shown as arrows.

When the linked list is created in a programming language, what will the integer value stored in a pointer represent?

- The algorithm will need to:

- You may wish to use a **Current Node Pointer** and a **Previous Node Pointer** in your response.

This image shows a blank sheet of white paper with horizontal blue ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

(Total 14 marks)

Q25.

A computer program stores a list of integers in an array named `List`. The numbers in the array are to be sorted into ascending order so that a particular efficient search algorithm can be used to search for a number.

- (a) One of the search algorithms in **Table 1** can only be used successfully on a sorted list.

Place **one** tick next to the name of the algorithm that requires a list to be sorted.

Table 1

Algorithm Name	Requires Sorted List? (Tick one box)
Binary search	
Linear search	

(1)

- (b) The pseudo-code for a standard algorithm that can be used to sort the data in the array `List` into order is shown in **Figure 1**. The variable `ListLength` stores a count of the number of items in the array `List`.

Array indexing starts at 1.

Figure 1

```

For OuterPointer ← 2 To ListLength
    CurrentValue ← List[OuterPointer]
    InnerPointer ← OuterPointer - 1
    While InnerPointer > 0 And
        List[InnerPointer] > CurrentValue Do
        List[InnerPointer + 1] ← List[InnerPointer]
        InnerPointer ← InnerPointer - 1
    EndWhile
    List[InnerPointer + 1] ← CurrentValue
EndFor

```

Complete the empty (unshaded) cells in the trace table (**Table 2**) for an execution of the algorithm in **Figure 1** when the array `List` contains the values 9, 8, 5 and 6 in that order.

Table 2

List Length	Outer Pointer	Current Value	Inner Pointer	List			
				[1]	[2]	[3]	[4]
				9	8	5	6
4	2		1				

			0				
	3		2				
			1				
			0				
	4		3				
			2				
			1				

(3)

- (c) In the trace table (**Table 2**), when the variable `OuterPointer` contains the value 2 and then 3, the value of the variable `InnerPointer` decreases to 0. When `OuterPointer` contains 4, `InnerPointer` stops decreasing when it reaches the number 1.

Explain why `InnerPointer` does not decrease to 0 when `OuterPointer` contains 4.

(1)

- (d) Tick **one** box in **Table 3** to indicate the correct Order of **Time** Complexity of the standard algorithm in **Figure 1**.

Table 3

Order of Time Complexity	Tick one box
$O(n)$	
$O(n^2)$	
$O(2^n)$	

(1)

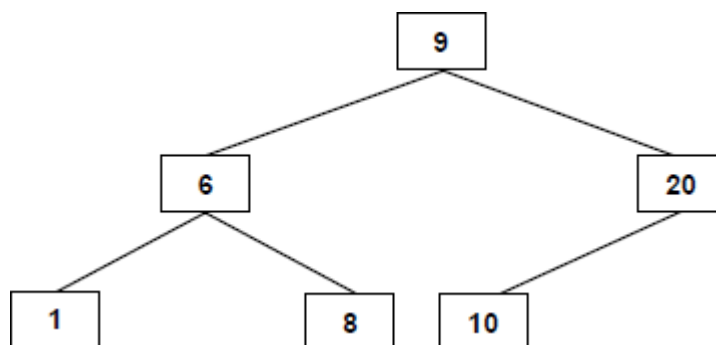
- (e) State the name of the standard algorithm that is represented by the pseudo-code in **Figure 1**.

(1)

- (f) Instead of storing a list of numbers in an array as in (b), the numbers could be stored in a binary search tree. This would also enable efficient searching.

The numbers 9, 6, 1, 8, 20 and 10 are put into a binary search tree in that order. **Figure 2** shows this binary search tree.

Figure 2



- (i) A search of the binary tree is performed for the number 8.

List the numbers, in the order that they would be checked, for the search to determine that the number 8 **is present** in the tree.

(1)

- (ii) A search of the binary tree is performed for the number 11.

List the numbers, in the order that they would be checked, for the search to determine that the number 11 **is not present** in the tree.

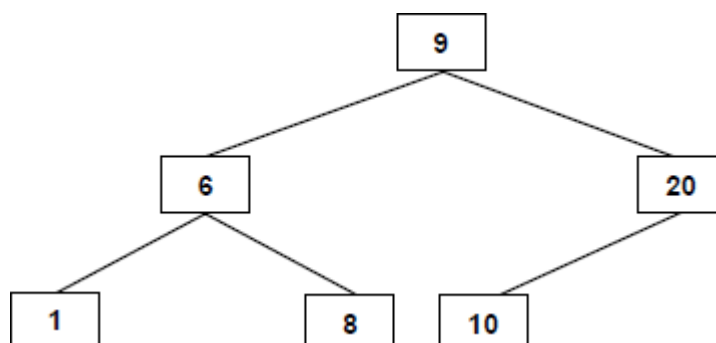
(1)

- (g) The numbers 4, 5 and 3 are to be added into the binary search tree, in that order.

Figure 3 below is an identical copy of **Figure 2**.

Complete **Figure 3** below to show the binary search tree from **Figure 2** after the extra numbers have been added into it.

Figure 3



(2)

(Total 11 marks)

Q26.

A computer program is being developed to play a card game on a smartphone. The game uses a standard deck of 52 playing cards, placed in a pile on top of each other.

The cards will be dealt (ie given out) to players from the top of the deck.

When a player gives up a card it is returned to the bottom of the deck.

- (a) Explain why a queue is a suitable data structure to represent the deck of cards in this game.

(1)

- (b) The queue representing the deck of cards will be implemented as a **circular** queue in a fixed size array named `DeckQueue`. The array `DeckQueue` has indices running from 1 to 52.

The figure below shows the contents of the `DeckQueue` array and its associated pointers at the start of a game. The variable `QueueSize` indicates how many cards are currently represented in the queue.

DeckQueue

Index	Data
[1]	10-Hearts
[2]	2-Spades
[3]	King-Hearts
[4]	Ace-Clubs
.	
.	
.	
[52]	8-Diamonds

FrontPointer = 1

RearPointer = 52

QueueSize = 52

- (i) Ten cards are dealt from the top of the deck.

What values are now stored in the `FrontPointer` and `RearPointer` pointers and the `QueueSize` variable?

`FrontPointer` = _____ `RearPointer` = _____

`QueueSize` = _____

(1)

- (ii) Next, a player gives up two cards and these are returned to the deck.

What values are now stored in the `FrontPointer` and `RearPointer` pointers and the `QueueSize` variable?

`FrontPointer` = _____ `RearPointer` = _____
`QueueSize` = _____

(1)

(iii) Write a pseudo-code algorithm to deal a card from the deck.

Your algorithm should output the value of the card that is to be dealt and make any required modifications to the pointers and to the `QueueSize` variable.

It should also cope appropriately with any situation that might arise in the `DeckQueue` array whilst a game is being played.

(6)

(c) The program for the card game will be an event-driven program.

Explain what it means for a program to be described as event-driven.

(2)

(d) The card game program will interact with the operating system on the smartphone.

Describe **two** differences between the operating system that is installed on the smartphone and an operating system that would be used on a desktop computer.

Difference 1 _____

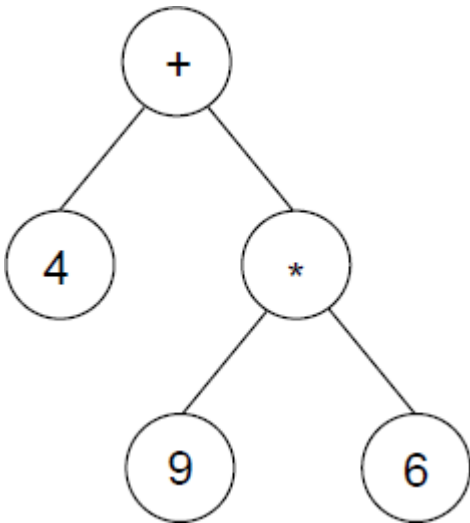
Difference 2

(2)

(Total 13 marks)

Q27. A tree can be used to represent a mathematical expression. This is known as an expression tree. **Figure 1** is an expression tree for the infix expression $4 + 9 * 6$.

Figure 1



- (a)

An expression tree is an example of a rooted tree.

State the contents of the root node: _____

List the contents of **all** of the leaf nodes: _____

(2)
- (b)

The expression tree in **Figure 1** could be represented using three one-dimensional arrays named **A**, **B** and **C**. **Figure 2** shows a representation of **Figure 1** together with the array indices.

Figure 2
Arrays

Index	A	B	C
[1]	+	2	3
[2]	4	0	0
[3]	*	4	5

[4]	9	0	0
[5]	6	0	0

Describe the role of each of the arrays **A**, **B** and **C**.

A:

B:

C:

(3)

- (c) What does an entry of 0 in array **B** indicate?

(1)

- (d) The procedure in **Figure 3** describes a type of tree traversal that can be carried out on the representation of the tree shown in **Figure 2**.

Figure 3

```

Procedure Traverse(Pos:Integer)
    If B[Pos] > 0 Then Traverse(B[Pos])
    If C[Pos] > 0 Then Traverse(C[Pos])
    Output A[Pos]
End Procedure

```

Using the table below, trace the execution of the procedure when it is called using `Traverse(1)`. You may not need to use all of the lines provided in the table.

Pos	Output

(4)

- (e) Which type of tree traversal does the procedure `Traverse` carry out?

(1)

- (f) What does the output of the procedure represent?

(1)

(Total 12 marks)

Mark schemes

Q1.

Repeat
 $S \rightarrow Q$
Until Q empty
Queue emptied to a stack
Elements taken from front of queue and placed / pushed on stack

2 marks

Repeat
 $Q \rightarrow S$
Until S empty
Stack emptied to a queue
Elements popped/taken from top of stack placed in queue

2 marks

Or suitable diagram

1 mark

[4]

Q2.

(a) An abstraction / leaving out non-essential details // A mathematical representation of reality;

1

(b) *1 mark for naming or describing **two** pointers from this list:*

- Front/start/head pointer
- Next node pointer
- Previous node pointer
- Rear/end/tail pointer
- R** Next free space pointer

*1 mark for stating the purpose of **one** of the pointers that have been named:*

- (Front/start/head pointer) to indicate where to remove items from // who should be served next // who is currently being served;
NE to point to start of list
- (Next node pointer) to link items in list together // to show order of list // so items can be inserted into middle of list // to traverse list;
- (Previous node pointer) to link items in list together // to show order of list // so items can be inserted into middle of list // to traverse list backwards;
- (Rear / end / tail pointer) to indicate where to add new items to // so new people can be added to queue
NE to point to end of list
A Contextualised answers which refer to queue instead of list or adding people to a queue.
R Answers which clearly relate to the use of a fixed-size array.

2

(ii) Priority (queue);

1

- (c) Allow any reasonable example that would require randomness e.g. time next person joins queue, inter-person arrival time, time to be served, choice of meal, type (student / teacher) of next person to arrive;

R number of students / teachers / people in queue

1

[5]

Q3.

- (a) Structure shown correctly with data in consecutive locations, and start and stop pointers

Must have data in boxes or a label

3

- (b) Procedure insert

{Check for overflow}

If (Start pointer=1 and) stop pointer = max size

2 marks

(or Start pointer = stop pointer+ 1) then

Report queue full

End

Endif

1 mark

Parts in brackets refer to circular queue not essential. 2 marks for checking. 1 mark for an attempt to check. 1 mark for stopping if full

{Insertion of data}

(If stop pointer = maxsize then

Stop pointer:= 1

Else)

Stop pointer:=stop pointer+ 1

(Endif)

queue(stop pointer):=data

endproc

1 for changing stop pointer. 1 for inserting data

2 marks

5

[8]

Q4.

- (a) Temp $\leftarrow$ Front;
Front $\leftarrow$ Temp.Next//Front $\leftarrow$ Temp^.Next;
Dispose (Temp); **A** Free(Temp)

Alternative

Temp $\leftarrow$ Front.Next// Temp $\leftarrow$ Front^.Next;

Dispose (Front); A Free(Temp)
Front ← Temp;

3

(b) AddItem//Add;

1

(c) (i) Full/FullQueue;

1

(ii) No memory used for pointers;
I Faster
R Easier to program

1

(iii) Size is limited by array size;
memory wasted when not full;

2

[8]

Q5.

(a) (i) First In First Out;
or by description

(ii) Last In First Out;
or by description

2

(b)

	FIFO	LIFO
Queue	✓	
Stack		✓

2

(c) Reverse the contents of a queue/list;
Push all contents of queue/list onto stack then pop them off into a new queue/list;
Procedure/function calls;
Local variables;
Parameters;
Return Address;
Volatile environment; **A** register contents State 1 Describe 1

2

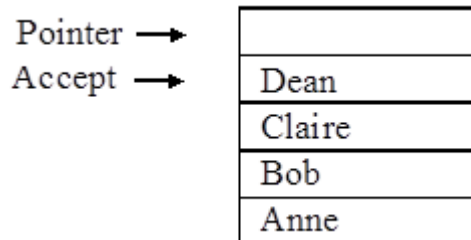
(d) List of elements inserted into tree;
To allow rapid/fast searching of the data;
To output sorted/ordered data;

2

[8]

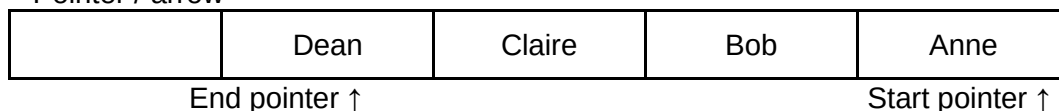
Q6.

(a)



2

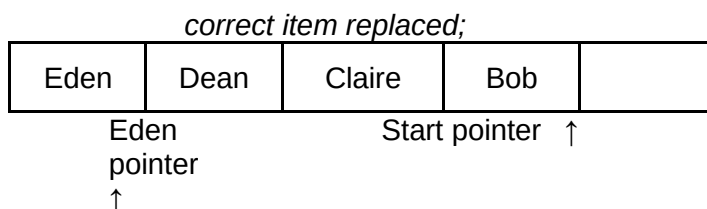
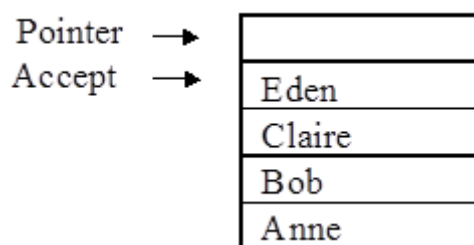
Dean accessed first
Pointer / arrow



Anne accessed first;
Named pointers correct;

2

(b)



Correct item retrieval & input;
Correct moving of pointers

3

- (c) In a linear queue data is static, so queue 'moves' through storage/
In a FIFO structure storage locations are only used once;(1)
In a circular queue, the locations will be re-used;(1)
Thus a circular queue has a more efficient use of memory;(1)

1 mark for each of 2 points

2

[9]

Q7.

- (a) (i) Empty entries waste space // Maximum/fixed/static size
A stack may overflow

1

- (ii) Space used by pointers // more complex to program; 1
- (b) (i) The size of the stack /amount of data is known/limited/predictable
Memory saved since no pointers (if not given in a (ii))
R easier to program 1
- (ii) The size of the stack is unknown//
The stack is volatile/ number of items fluctuates widely; 1
- [4]**

Q8.

- (a) (i) • poorly structured code;
• uses GoTo statements;
• the flow of control jumps out of a loop;
• nothing reported to the user when no matching name found;
• abbreviated variable for 'position' variable;
• ReadLn is better than Read;
• Program only iterates once / considers only the first array element;
• (if duplicates) only the first matching surname is found;
• (loop terminates at 20) does not allow for additional array /name entries;
A poor layout - excessive indentation used;
I. variable declaration // reference to the syntax Max 2
- (ii) All statements must have correct identifier name correct data type
(String / Text // Integer / Byte / Word / Int / Shortint / Short as appropriate)

In addition, either array must have brackets to indicate an 'array' 19/20
to indicate a range; Max 2
- (b) Initialisation of counter or Boolean variable
P := 1 / P := 0 / For P := 1 to 20 // IsFound := False;
- Looping
LOOP UNTIL // DO WHILE // WHILE DO // REPEAT UNTIL and used at the beginning/end of a code block as appropriate;
- Some loop condition is met
(P = 20/21) OR IsFound = TRUE / P = 20/21 // IsFound = TRUE / IsFound;
- IF with use of the array
IF NoOfClaims [P];
- Selection condition
>4 / >=5;
- Loop counter incremented
P = P+1
- Final output

Correct logic followed with OUTPUT 'Yes'

A multiple times

Final output

Correct logic followed with OUTPUT 'No'

R Multiple times

R 'Prose' scores 0

5

[9]

Q9.

- (a) Tail(Ports) - [Barcelona, Athens, Alexandria, Tunis, Lisbon]

square brackets needed

1

Head(Tail(Tail(Ports))) – Athens (2)

[Athens] (1)

2

Empty(T(T(T(T(T(T(Ports))))))) – True (2)

True [] (1)

[True] (0)

2

- (b) **Recursively defined**

A definition which is defined in terms of itself/contains within its body a reference to itself/calls itself ;

A re-entrant; (In specimen papers 2001/2, but refers specifically to a procedure)

1

- (c) **Stack necessary**

The state of the machine/contents of appropriate registers/ return address // saved each time the procedure is called (1) and retrieved in reverse order from the stack as control is progressively returned (1)

OR

Different value of parameters /local variables (1) must be available each time procedure is called (1)

OR

P must be re-entrant (In specimen papers 2001/2)(2)

2

- (d) Lisbon first (1)
Southampton last(1)
All 6 in order (1)
No punctuation (1)

i.e. Lisbon Tunis Alexandria Athens Barcelona Southampton;

4

- (e)

1	Southampton	5	Head Pointer 4
2	Barcelona	6	
3	Athens	2	
4	Alexandria	3	
5	Tunis	0 - terminator	
6	Lisbon	1	

2

[14]

Q10.

- (a) Next item to be added is at position/location/address (Tail + 1);
Position/location/address Tail is the last item in the queue ;
R 'points to the end of the queue'

Max 1

- (b) Cat // item at position Head ;

1

- (c)

	6	
Tail	→ 5	'Shark'
	4	'Eel'
	3	'Snake'
Head	2	'Frog'
	1	'Dog'
	0	'Cat'

Snake + Eel + Shark at positions 3,4,5 ;(1)

Tail points to 5 ;(1)

Head points to 2 ;(1)

I. Dog and Cat crossed through

3

- (d) Tail will eventually reach position 99 (**A** 100) ;
Head will eventually reach 99 (**A** 100);
Memory/queue will become full ;
Space is not re-useable ;

Max 2

[7]

Q11.

- (a) Salesperson 7;
April /month 4;
The number of storecards 'taken out';
Max 2
- (b) StoreCards + sensible subscripts [1..10, 1..6] / (1 to 10, 1 to 6) / [0..10, 0..6] /
(0 to 10, 0 to 6) / (10,6) / [10] 6];
StoreCards + Integer / Byte;
2
- (c) StoreCards (8, 1);
= 13 / := 13 / ← 13;
Must be an assignment statement
2
- (d) Key in / Input the employee number; the program calculates the total number
of store cards for a single person // print/outputs/displays the total for a single
person; over six months;
Max 2
- (e) (i) Single / real / float;
R Floating point / Double
1
- (i) Boolean /Yes-No / True-False; R Y/N / T/F
1
- (iii) Integer/ byte;
1
- [11]

Q12.

Marks are for AO1 (understanding)

Static data structures have storage size determined at compile-time / before
program is run / when program code is translated; dynamic data structures can
grow/shrink during execution / at run-time;

//

Static data structures can waste storage space/memory if the number of data items
stored is small relative to the size of the structure; whereas dynamic data structures
only take up the amount of storage space required for the actual data;

//

Static data structures have fixed (maximum) size; whereas size of dynamic data
structures can change;

//

Dynamic data structures (typically) require memory to store pointer(s) to the next
item(s); which static data structures (typically) do not need; **NE**. Dynamic data
structures use pointers

//

Static data structures (typically) store data in consecutive memory locations; which
dynamic data structures (typically) do not;

[2]

Q13.

- (a) (i) 271; 1
- (ii) The required item might be the 271st one/last one/ not be present//
Every item accessed; 1
- (b) (i) 9; 1
- (ii) Each comparison halves the number of items to be accessed//271 lies
between 2^8 and 2^9 ; 1
- (c)

Count1	Count2	Temp	A				
			[1]	[2]	[3]	[4]	[5]
-	-	-	23	45	16	12	31
1	1						
	2	45		16	45		
	3	45			12	45	
	4	45				31	45
2	1	23	16	23			
	2	23		12	23		
	3						
	4						
3	1	16	12	16			
	2						
	3						
	4						
4	1						
	2						
	3						
	4						

1 mark for Count1
1 mark for Count2
1 mark for Temp

(ii) (bubble) sort the items into ascending order;

(iii) Reduce the number of tests each pass// stop when no swaps occur during a pass//Add a flag No Swaps to indicate when no swaps occur// change loop control to Repeat until no swaps// sort variable sized array;

5

1

1

[11]

Q14.

- (a) A procedure that is defined in terms of itself;
A A procedure that calls itself
R re-entrant

1

- (b) Store return addresses;
 Store parameters;
 Store local variables/ return values;

Max 1

- (c)

Number	Entry	Output
11	1	
11	2;	
11	3;	
11	4;	4;

4

- (d) A linear search//
 To find/output the position/index of Number in Items;

1

- (e) Number is not an entry in Items// Stack overflows;

1

- (f) Test for reaching the end of Items;

1

- (g) Binary Search;
 An iterative solution;

Max 1

[10]

Q15.

- (a) Last (item) in, is the first (item) out / first (item) in is the last (item) out ;
R LIFO / FILO

1

- (b) (i)

600	'A'
601	'V'
602	'E'
603	'R'

604	'Y' ;
605	

All items in the correct locations

1

(ii)

599	
600	'A'
601	'V'
602	'E' ;
603	
604	
605	

Correct three items // ft from an incorrect (i) including 605 as the first location used ;

A 'R' and 'Y' entries indicated in some way as 'deleted'

1

(iii)

600	'A'
601	'V'
602	'E'
603	'S'
604	'P' ;
605	

Correct list of five items // ft from an incorrect (i) + a correct ft (ii) including 605 as the first location used ;

1

- (c) (i) Queue ;
A First In – First Out FIFO / LILO

1

- (ii) Items are removed/popped from the stack (one at a time) (and items are then added to the queue);

1

- (iii) Items leave the queue on a 'first in-first out' basis ; **A** from the front of the queue

1

- (iv) 'Y', 'R', 'E', 'V', 'A' on the queue ;
 Y', 'R', 'E', 'V', 'A' on the final stack ;
 A using 701 for the first queue location

2

[9]

Q16.

- (a) 2-D array;

1

- (b) Shows that sales person 2; did meet their target; for Quarter 3 /July – September;

Max 1

- (c)

Person	Quarter	Target [Person, Quarter]	NewArray			
			[1]	[2]	[3]	[4]
1	1, 2, 3, 4		0	0	0	0;
1	1	Y				
	2	N		1;		
	3	Y				
	4;	N				1
2	1	N	1			
	2	N		2		
	3	Y				
	4	Y				
3;	1	N	2;			
	2	N		3		
	3	N			1	
	4	N				2;

NewArray initial values all 0;

1

Person loop counter 1 to 3;

1

Person 1 – is followed by quarters 1 to 4 in sequence;

1

NewArray[2] = 1 for person = 1 and Quarter = 2;

1

Final NewArray[1] = 2;

1

Final NewArray[2 and 3 and 4] values are correct;

1

- (d) Stores the (total) number of sales staff who did not meet their target //

the(total) number of sales targets not met;
for each quarter;

2

[10]

Q17.

- (a) (i) Empty entries waste memory // Maximum size// fixed size;
- (ii) Memory used by pointers//takes more time to add / delete nodes//
indirect access takes more time;
R programming difficulties
- (b) Place next item in first location/ location 0/ location 1//
Implement a circular array/queue // allow wraparound;
- (c) IsFull/IsQueueFull;

1

1

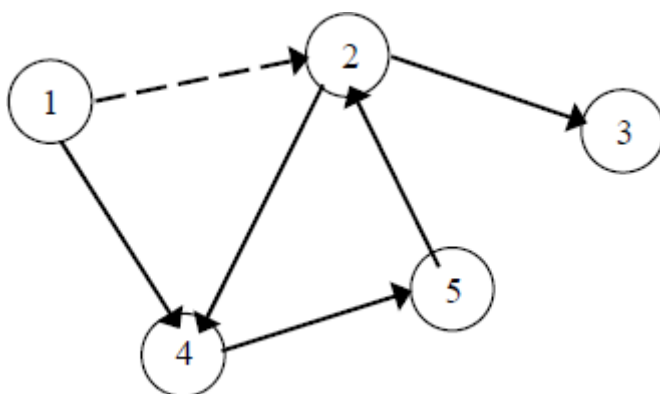
1

1

[4]

Q18.

(a)



1 mark for all 5 lines correctly drawn

1 mark for all 5 arrowheads pointing in correct directions

Max 1 if more than 5 lines drawn by candidate (note that dotted arrow is given in question)

A arrowheads at any position on line

2

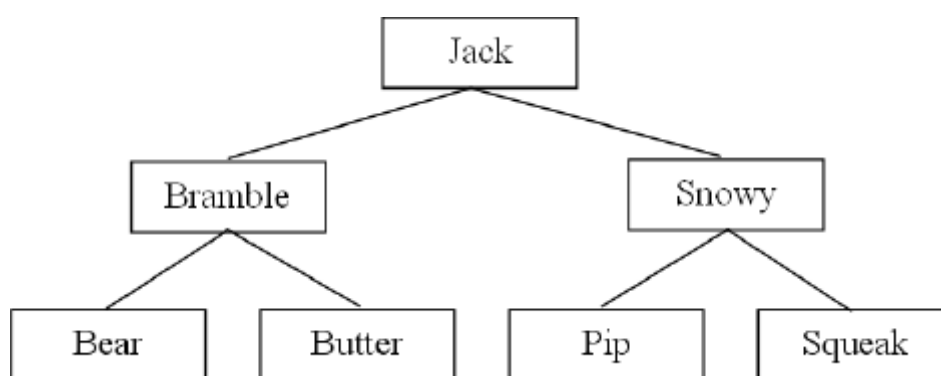
- (b) Adjacency matrix appropriate when there are many edges between vertices // when edges may be frequently changed // when presence/absence of specific edges needs to be tested (frequently)
Adjacency list appropriate when there are few edges between vertices // when graph is sparse // when edges rarely changed //when presence/absence of specific edges does not need to be tested (frequently)
A alternative words which describe edge e.g. connection, line

2

- (c) Connected // There is a path between each pair of vertices;
 Undirected // No direction is associated with each edge;
 Has no cycles // No (simple) circuits // No closed chains // No closed paths in which all the edges are different and all the intermediate vertices are different // No route from a vertex back to itself that doesn't use an edge more than once or visit an intermediate vertex more than once;
Alternative definitions:
 Graph with no cycles, and a simple cycle is formed if any edge is added to it;;
 Graph which is connected, and it is not connected anymore if any edge is removed from it;;
 Graph in which any two vertices can be connected by a unique simple path;;
 (Note: not just adjacent vertices)
 Graph which is connected and has $n - 1$ edges where n is no of vertices;;
 Graph which has no simple cycles and has $n - 1$ edges where n is no of vertices;;

Max 2

(d)



1 mark for Jack as root

1 mark for Bramble and Snowy as children of Jack

1 mark for four correct children of Bramble and Snowy

DPT if arrows drawn instead of lines

DPT if any node has more than 2 child nodes

A "mirror image" answers which are consistent.

3

(e) **For solution with 3 arrays:**

One array stores data items;

One array for left child pointers;

One array for right child pointers;

Pointers stored at same location in arrays as corresponding data item;

For solution with 1 array of records:

Record created to store data item and pointers;

One pointer to left child;

One pointer to right child;

For either of the above solutions:

Rogue value (allow example) used to indicate no child;

Variable indicates position in array(s) of root node // Root node stored at first location/start of array(s);

If answered as diagram:

Column for data with at least three correct data items in it;

Use of rogue value for a node that does not have child;

Correct value for a start pointer variable indicating position of root node in the array (not drawn as an arrow, array indices must be labelled);

Column for left child pointers*;

Column for right child pointers*;

* = To get these marks, there must be a sufficient number of pointers to demonstrate that the data structure is a representation of a binary tree, but it is not necessary for every item to be shown. Also the array indices must be shown.

Max 3

[12]

Q19.

(a)

ListLength	New	P	q	List				
				[1]	[2]	[3]	[4]	[5]
4	38	-	-	9	21	49	107	
		1						
		2						
		3						
			4					107
			3				49	
						38		
5								

4,5 in sequence for ListLength;

1,2,3 in sequence for p;

4,3 in sequence for q;

Final list in array is 9, 21, 38, 49, 107;

Do not award a mark if additional values indicated e.g. 4 for p

4

(b) Inserts an item/variable New into list at correct position/preserving order//into sorted list (or equivalent);

1

(c) (i) Static structures have fixed (maximum) size whereas size of dynamic structures can change // Size of static structure fixed at compile-time whereas size of dynamic structure can change at run-time;
Static structures can waste storage space/memory if the number of data items stored is small relative to the size of the structure whereas dynamic structures only take up the amount of storage space required for the actual data;
Dynamic data structures (typically) require memory to store pointer(s) to the next item(s) which static structures do not need;
A just one side of points, other side is by implication

Max 1

- (ii) Heap is pool of free/unused/available memory;
Memory allocated/deallocated at run-time (to dynamic data structure);

Max 1

[7]

Q20.

(a)

Reverse Polish Notation	Equivalent Infix Expression
45 6 +	45 + 6 <i>R</i> 6 + 45
12 19 + 8 *	(12 + 19) * 8 <i>R</i> 12+19*8, (19+12)*8 <i>A</i> x for *

2

- (b) Simpler for a machine / computer to evaluate // simpler to code algorithm
A easier **R** to understand
Do not need brackets (to show correct order of evaluation/calculation);
Operators appear in the order required for computation;
No need for order of precedence of operators;
No need to backtrack when evaluating;
A RPN expressions cannot be ambiguous as **BOD**

1

(c)

String Pos	Token	Integer Val	Op1	Op2	Result	Stack
0	-	-	-	-	-	
1	6	6				6
2	4	4				4 6
3	+		6	4	10	10
4	3	3				3 10
5	2	2				2 3 10
6	+		3	2	5	5 10
7	*		10	5	50	50

Output : 50

1 mark for each of rows 1–3

1 mark for rows 4 and 5 together

1 mark for rows 6 and 7 together

1 mark for correct final output

Values of Op1 and Op2 MUST be assigned in rows 3, 6 and 7 to award the marks for these rows. They cannot be inferred from incorrectly entered previous values.

I values in empty cells, even if they are incorrect.

6

(d) If StackArray is full
 Then Stack Full Error
 Else
 Increment TopOfStackPointer
 StackArray [TopOfStackPointer] ← ANumber
 EndIf

1 mark for appropriate *If* structure including condition (does not need both *Then* and *Else*) – Do not award this mark if ANumber is put into StackArray outside the *If*.

1 mark for reporting error in correct place

1 mark* for incrementing TopOfStackPointer

1 mark* for storing value in ANumber into correct position in array

* = if the store instruction is given before the increment instruction OR

the *If* structure then award **Max 1** of these two marks **UNLESS** the item is inserted at position *TopOfStackPointer+1* so the code would work.

I initialisation of *TopOfStackPointer* to 0

A *TopOfStackPointer=20/>=20* for Stack is full

A Logic of *if* structure reversed i.e. If stack is not full /

TopOfStackPointer<20 / <>20/!=20 and Then, Else swapped

A Any type of brackets or reasonable notation for the array index

DPT If candidate has used a different name any variable then do not award first mark but award subsequent marks as if correct name used.

Refer answers where candidate has used a loop to find position to insert item into stack to team leaders.

4

[13]

Q21.

(a)

ListLength	New	p	q	List				
				[1]	[2]	[3]	[4]	[5]
4	48	–	–	19	43	68	107	
		1						
		2						
		3						
			4					107
			3				68	
						48		
5								

1 mark: 5 is the only value written in the *ListLength* column

1 mark: 1, 2, 3 in that order are the only values written in the *p* column

1 mark: 4, 3 in that order are the only values written in the *q* column

1 mark: final list is: 19, 43, 48, 68, 107

Ignore values being repeated unnecessarily in trace

MAX 3 if any incorrect values in table

4

(b) Inserts an item/variable *New* into list at correct position/preserving order//into sorted list (or equivalent);

1

(c) Heap is (pool of) free/unused/available memory;
Memory allocated/deallocated at run-time (to/from dynamic data structure(s));

Max 1

[6]

Q22.

(a) It hides the detail of how the list will be stored/implemented

from the programmer // a programmer working on the rest of the program does not need to know how the `LinkedList` class works // a programmer working on the rest of the program needs only concern themselves with the interface to the `LinkedList` class;

A. "user" for "programmer" as BOD mark

1

- (b) The procedures/functions are public as programmer (writing the rest of the program) will need access to the operations defined in the procedures and functions from outside of the class / elsewhere in the program (so they must be public); **A.** just one of procedures or functions **A.** Procedures/functions will be accessible

The data items are private to prevent them being changed directly from outside of the class // to avoid the integrity of the data structure being damaged / changed accidentally (from outside the class); **A.** "elsewhere in program" for "outside of the class"

So that the implementation of `LinkedList` can be changed and programs written using only the public functions and procedures will still work;

MAX 2

2

- (c) **OVERALL GUIDANCE:**

Solutions should be marked on this basis:

- Up to **5 marks** for correctly locating the position to delete the item from.
- Up to **3 marks** for deleting the item and updating pointers as required.

The addition of any unnecessary steps that do not stop the algorithm working should not result in a reduction in marks.

Responses should be accepted in pseudo-code or structured English but not in prose.

If you are unsure about the correctness of a solution please refer it to a team leader.

SPECIFIC MARKING POINTS:

Correctly locating deletion point (5 marks):

1. Initialising `Current` to `Start` before any loop;
2. Use of loop to attempt to move through list (regardless of correct terminating condition);
3. Advancing `Current` within loop;
4. Correctly maintaining the `Previous` pointer within loop;
5. Sensible condition to identify position to delete from (suitable terminating condition for loop);

Correctly deleting item (3 marks):

6. Update `Next` pointer of node before node to delete to

- point to node after it;
- 7. Test if item to delete was first item in list, and if so update `Start` pointer instead of `Next` pointer of node before the one to delete;
- 8. Release the memory used by the item being deleted back to the operating system;

Mark point 2 should be awarded if, within the loop, `Current` is being changed (even if not correctly changed).

Mark point 4 can be awarded if `Previous` is set to `Current` before `Current` is changed, even if `Current` is not being correctly updated.

Mark point 5 can be awarded if there is a sensible condition, even if `Current` is not correctly updated.

Mark point 6 can be awarded even if the value of `Previous` was not correctly maintained in the loop.

Mark points 6 and 7 can only be awarded if `Current` has not already been released (or attempted to be released).

Mark point 8 should only be awarded if this is done after and a loop to search for the item to delete, regardless of whether or not the correct item would be found or if it is done inside the loop but also within an if statement that correctly identifies the item to delete.

A. Deletion takes place inside of loop if the correct item to delete had been identified with an if statement and the loop will be exited at some point after deletion.

A.. Use of any type of condition controlled loop, as long as logic is correct.

A. Use of alternative variable names and instructions, so long as the meaning is clear.

A. Use of clear indentation to indicate start/end of iteration and selection structures.

A. Responses written in structured English, so long as variable names are used and the descriptions of what will be done are specific.

A. Use of Boolean variable to control loop as long as it is set under the correct conditions and has been initialised.

R. Responses written in prose.

R. Do not award mark points if incorrect variable names have been used, but allow minor misspellings of variable names.

EXAMPLE SOLUTIONS:

The examples below are complete solutions that would achieve full marks. **Refer recursive solutions to Team Leaders.**

Example 1

```
If Start.DataValue = DelItem Then
    Start ← Start.Next
```

```

    Release(Start)
Else
    Current ← Start
    Repeat
        Previous ← Current

        Current ← Current.Next
    Until Current.DataValue = DelItem
    Previous.Next ← Current.Next
    Release(Current)
EndIf

```

Example 2

```

Current ← Start
While Current.DataValue ≠ DelItem
    Previous ← Current

    Current ← Current.Next
EndWhile
If Current = Start Then
    Start ← Current.Next
Else
    Previous.Next ← Current.Next
EndIf
Release(Current)

```

Example 3

```

If Start.DataValue = DelItem Then
    Start ← Start.Next
    Release(Start)
Else
    Deleted ← False

    Current ← Start
    While Deleted = False
        If Current.DataValue = DelItem Then
            Previous.Next ← Current.Next
            Release(Current)

            Deleted ← True
        Else
            Previous ← Current

            Current ← Current.Next
        EndIf
    EndWhile
EndIf

```

Q23.

- (a) **Implementation One** would need to use a linear search // would need to look at every word in the array (before the one that is being searched for) // lookup time is proportional to number of words in list // lookup is $O(N)$; **N.E.** "search" without further clarification that this would be linear
Implementation Two would use the hash function/hashing

to directly calculate where the word would be stored // could jump directly to the correct position/location/index for the word in the array // lookup time is constant regardless of how many words in list // lookup is $O(1)$; **A.** No need to go through words in list

2

- (b) The (record for) each word/both words would be stored at the same position/index/location in the array;
A. The second word would be stored over/replace the first;
N.E. A collision has occurred

Store record/word in the next available position in the array // store a pointer (in each array position) that points to a list of records that have all collided at the position // rehash the word;

A. Idea that each array position could store more than one record e.g. five records per location, if explained.

A. Example of what “next available” might be.

R. The use of a different hashing function at all times ie not just rehashing.

2

- (c) The hash function could compute the same value/location for more than one/two English word(s), so need to verify if the English word stored at the location is the one that is being looked up;
 To avoid returning a French translation that is for a different English word, which is stored at the same location as the word that is being looked up // if a collision occurred (when storing the words) it will not be possible to tell if the translation is correct;

A. More than one word could be stored in each location

R. So that French to English translation can be done

MAX 1

1

[5]

Q24.

- (a) Static structures have fixed (maximum) size whereas size of dynamic structures can change // Size of static structure fixed at compile-time whereas size of dynamic structure can change at run-time;
 Static structures can waste storage space / memory if the number of data items stored is small relative to the size of the structure whereas dynamic structures only take up the amount of storage space required for the actual data;
 Dynamic data structures (typically) require memory to store pointer(s) to the next item(s) which static structures (typically) do not need // Static structures (typically) store data in consecutive memory locations, which dynamic data structures (typically) do not;
Max 2
A just one side of points, other side is by implication
NE Dynamic data structures use pointers

2

- (b) Not possible to simply insert item into middle of list;

Must move all items that should come after the new process down in the array;
NE move all data
Moving items is time consuming;
In a dynamic implementation, insertion achieved by adjusting pointers;
Max 2

2

(c) Priority (queue);

1

(d) (i) Memory allocated / deallocated at run-time / for new items (to dynamic data structure);
(Provides a) pool of free / unused / available memory;
NE to store new items
Max 1

1

(ii) (Memory) address // memory location // position in memory;
NE position or location without reference to memory
R index

1

(iii) **OVERALL GUIDANCE:**

Solutions should be marked on this basis:

- Up to **4 marks** for correctly locating the position to insert the new process at.
- Up to **4 marks** for creating a new node and storing the correct data into it and the associated pointers.
Some marks can be awarded for this even if the locating process is incorrect / missing.

The full 7 marks should only be awarded for a complete fully working solution. If any steps are missed out, then award a Maximum of six marks (Max 6).

The addition of any unnecessary steps that do not stop the algorithm working should not result in a reduction in marks.

Responses should be accepted in pseudo-code or structured English.
If you are unsure about the correctness of a solution please refer it to a team leader. Also, responses in prose should be referred to team leaders.

SPECIFIC MARKING POINTS

Correctly locating insertion point (Max 4):

- 1 Initialising current node pointer to start pointer;
- 2 Use of loop to attempt to move through list (regardless of correct terminating condition);
- 3 Advancing current node pointer within loop;
- 4 Correctly maintaining a pointer to the node before the position that the new node should be inserted at;
- 5 Sensible condition to identify place to insert (suitable terminating condition for loop or condition in selection statement);

Correctly inserting new process (Max 4):

- 6 Create a new node / obtain new node from heap;
- 7 Store new process name and priority (in new node);
- 8 Update NextNodePointer of node before newly inserted one to point to new node;
- 9 Set NextNodePointer of new node to point to node after it;

Mark point 2 can only be awarded if, within the loop, current node pointer is being changed (even if not correctly changed).

Mark point 4 can only be awarded if mark point 3 had been awarded.

Mark point 5 can be awarded if there is a sensible condition, even if current node pointer is not correctly updated.

Mark points 8 and 9 can only be awarded if the correct insertion point has been found.

For any solution:

A use of either while or repeat loops, as long as logic is correct.

A storage of values into new node in any order, and regardless of whether the node has been created or not.

A use of ^ symbol to indicate the value stored at an address referenced by a pointer, for example CurrentNodePointer^. Priority indicates the value stored in the Priority field of the node pointed to by the pointer CurrentNodePointer.

A use of alternative variable names so long as the meaning is clear.

EXAMPLE SOLUTIONS AND MARKS:

These four examples show where marks should be awarded in some possible solutions (subject to a maximum mark of 7):

Example 1:

```
CurrentNodePointer = StartPointer;
Repeat
    PreviousNodePointer =
        CurrentNodePointer;
    CurrentNodePointer =
        NextNodePointer of current node;
Until priority of process in current node < priority
of process to add
//
    priority = "Low";;
Create new node;
Store new process name (and priority)
    in new node;
New node's NextNodePointer = Next
NodePointer of item at position
    PreviousNodePointer;
NextNodePointer of item at position
    PreviousNodePointer = Address of new
node;
```

Example 2:

This is an alternative way of expressing Example 1:

- 1 Load the Start Pointer value into the Current Node Pointer;
- 2 Copy value from Current Node Pointer into Previous Node Pointer;
- 3 Set Current Node Pointer to Next Node Pointer of current node;
- 4 If the priority of the data item at the current node is higher than or the same as the priority of the process to be added; then go back to step 2;
- 5 Create a new node;
- 6 Store the name of the new process (and its priority) in the new node;
- 7 Copy value from Next Node Pointer of list entry at position stored in Previous Node Pointer into the Next Node Pointer of the new node;
- 8 Set the Next Node Pointer of the list entry at position stored in the Previous Node Pointer to point to the new node;

Example 3:

```
CurrentNodePointer = StartPointer;
Inserted = False
While; Inserted = False Do
    If Current Node's priority < new item
        priority // = "Low"; Then
        Create new node;
        Store new process name (and priority)
            in new node;
        New node's NextNodePointer =
            CurrentNodePointer;
        NextNodePointer of item at position
            PreviousNodePointer = Address of new
            node;
        Inserted=True
    End If
    PreviousNodePointer = CurrentNodePointer;
    CurrentNodePointer =
        NextNodePointer of current node;
End While
```

Example 4:

```
CurrentNodePointer = StartPointer;
While; not at end of list // While
    CurrentNodePointer <> Nil // While
        priority of process at CurrentNodePointer
        >= priority of process to add Do
            If Current Node's priority is the required
                priority // = "Normal"; Then
                LastNodeOfCurrentPriorityPointer =
                    CurrentNodePointer;
            End If
            CurrentNodePointer =
                NextNodePointer of current node;
```

```

End While
Create new node;
Store new process name (and priority)
  in new node;
New node 's NextNodePointer = Next
  NodePointer of item at position
  LastNodeOfCurrentPriorityPointer;
NextNodePointer of item at position
  LastNodeOfCurrentPriorityPointer = Address
  of new node;

```

7

[14]

Q25.

(a)

Algorithm Name	Requires Sorted List? (Tick one box)
Binary search	☒
Linear search	☐

1 mark for having a tick in the "Binary search" row.

A alternative indicators for tick eg "Yes"

A a tick for "Binary search" and a cross for "Linear search"

R answers where two ticks have been used.

1

(b)

				List			
List Length	Outer Pointer	Current Value	Inner Pointer	[1]	[2]	[3]	[4]
				9	8	5	6
4	2	8	1		9		
			0	8			
	3	5	2			9	
			1		8		
			0	5			
	4	6	3				9
			2			8	
			1		6		

Award **1 mark** for each of the highlighted rectangles which has the correct

values written in it in the unshaded cells.
 Accept responses in which correct values are unnecessarily written out again.
 Do not award a mark for any rectangle which has an incorrect value written in it.

3

- (c) The value being moved / CurrentValue / 6 does not need to be put at the start of the list // should be inserted at position 2 not position 1;
 Because the second condition (in the While statement) is not satisfied;
MAX 1

1

(d)

Order of Time Complexity	Tick one box
$O(n)$	
$O(n^2)$	✓
$O(2^n)$	

A alternative indicators instead of a tick eg a cross, Y, Yes
R responses in which more than one box is ticked

1

- (e) Insertion sort;
A Insert sort

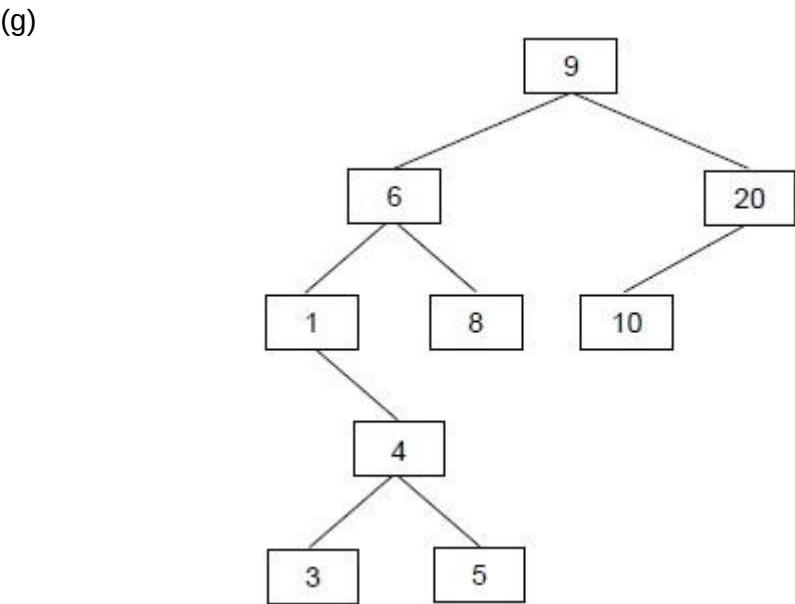
1

- (f) (i) 9, 6, 8;
 Must be in the order above. Can be separated by any character or a space

1

- (ii) 9, 20,10;
 Must be in the order above. Can be separated by any character or a space.

1



1 mark for inserting number 4 in the correct place
1 mark for inserting both numbers 3 and 5 in the correct place relative to 4
MAX 1 if any numbers added in the wrong place / any extra numbers added

2

[11]

Q26.

- (a) Values / cards need to be taken out of the data structure from the opposite end that they are put in // cards removed from top / front and added at end / bottom / rear;
 Values / cards need to be removed in the same order that they are added;
A It is First in First Out // It is FIFO;
A It is Last in Last Out // It is LILO;
MAX 1

1

- (b) (i) FrontPointer = 11
 RearPointer = 52
 QueueSize = 42
1 mark for all three values correct

1

- (ii) FrontPointer = 11
 RearPointer = 2
 QueueSize = 44

1 mark for all three values correct

A incorrect value for FrontPointer if it matches the value given in part (i) and incorrect value for QueueSize if it is equal to the value given for QueueSize in part (i) incremented by two (follow through of errors previously made)

1

- (iii) If DeckQueue is empty then
 Report error
 Else
 Output DeckQueue[FrontPointer]
 Decrement QueueSize
 Increment FrontPointer
 If FrontPointer > 52 Then
 FrontPointer = 1
 EndIf

1 mark for If statement to check if queue is empty – alternative for test is QueueSize = 0.

1 mark for reporting an error message if the queue is empty // dealing with the error in another sensible way – this mark can still be awarded if there is an error in the logic of the If statement, as long as there is an If statement with a clear purpose.

1 mark for only completing the rest of the algorithm if the queue is not empty – this mark can still be awarded if there is an error in the logic of the If statement, as long as there is an If statement with a clear purpose.

1 mark for outputting the card at the correct position

1 mark for incrementing FrontPointer and decrementing QueueSize

1 mark for If statement testing if the end of the queue has been reached

1 mark for setting FrontPointer back to 1 if this is the case – this mark can still be awarded if minor error in logic of If statement, eg >= instead of =

A `FrontPointer = (FrontPointer MOD 52) + 1` for **3 marks** or
`FrontPointer = (FrontPointer MOD 52)` for **2 marks**, both as alternatives
to incrementing and using and the second `If` statement - **deduct 1 mark**
from either of the above `if`
`QueueSize` has not been decremented
A any type of brackets for array indexing
I Additional reasonable `EndIf` Statements
MAX 5 unless all of the steps listed above are carried out

6

- (c) Flow of program / execution sequence determined by events // program
executes relevant code-handling block / procedure / sub-routine in response to
events;
Example of event such as clicking a button;
Message sent to program when event occurs;
System / message loop executes until application closes; this receives and
processes messages // use of event-listener / handler;
If several events occur they are queued;
MAX 2

2

- (d) The smartphone operating system:
- Will not have to support as wide a range of hardware devices /
peripherals // may not support external storage devices;
 - Will not / less likely to need to support the addition of new hardware
devices to the system;
 - Will have minimising power consumption as a higher priority; **A** will have
more sophisticated power management features
 - Will run application software in a sandbox // will (more tightly) restrict
access to resources by application software;
 - Must be capable of running on a device with less processing power //
less RAM / memory // smaller memory footprint;
 - Needs to work with specialised hardware devices eg GPS receiver,
accelerometer (**Note:** A relevant example must be given);

A Points made in reverse, but do not give **2 marks** for one point and its
reverse.
MAX 2

2

[13]

Q27.

- (a) +;
4, 9, 6; (in any order)
- (b) **A** Store the data / value (in the vertices / nodes);
A: holds the expression
B: Left pointer // points to the left child / left sub tree;
C: Right pointer // points to the right child / right sub tree;
A "indicates", "index" or other synonym for "points" / "pointer"
R Stores left / right subtree
- (c) The node has no left child / sub tree;

2

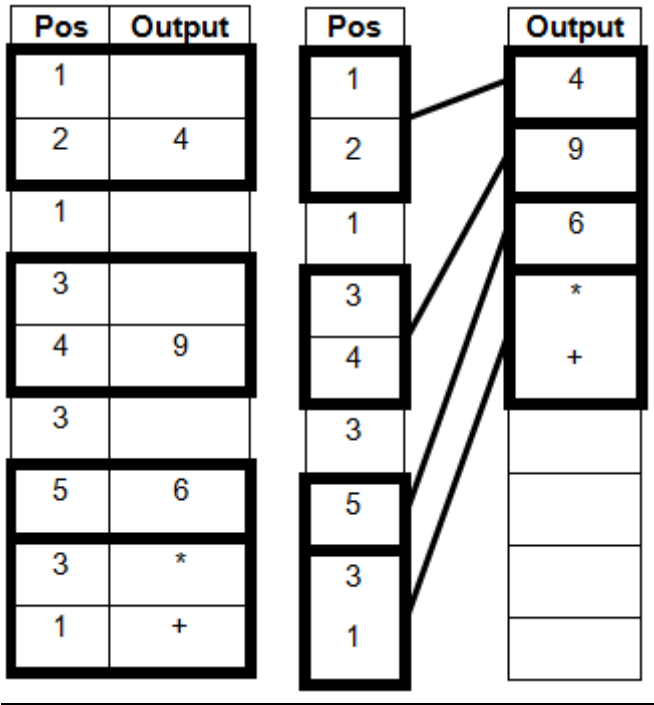
3

A there is nothing to the left
A this is a null pointer

1

- (d) One mark for each area outlined with a dark rectangle. Lines that are not outlined can be missed out.

Alternative 1Alternative 2



Mark against whichever alternative gives the highest mark.

Stop marking as soon as incorrect output is given.

4

- (e) Post-order;
A Depth-first
A Depth-first search as BOD
TO Depth-first pre / in-order

1

- (f) (4 + 9 * 6 in) Reverse Polish (Notation) // Postfix (Notation) // RPN;

1

[12]